

Lab Experiment: Order Processing & Class Relationships

Implementation of Inheritance, Subtype Polymorphism, and Association in Java

Course: Object-Oriented Programming Lab **Language:** Java (JDK 8+) **Document Code:** LAB-PRG-04

1. Objective

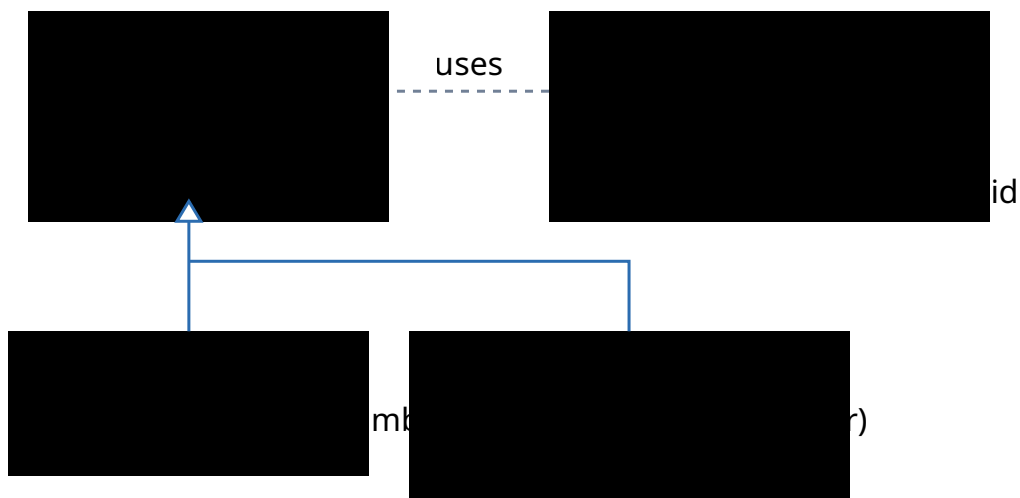
To implement an Object-Oriented Java application (LabProgram4) demonstrating class hierarchy, inheritance, subclassing, parameter passing via base-class types (polymorphic parameter binding), and class associations. Key concepts include:

- **Superclass Construction Chaining:** Utilizing `super(date, number)` to correctly forward variable parameters from child constructors to base class constructors.
- **Subtype Polymorphism:** Passing subclass instances (`specialorder`, `normalorder`) to methods accepting a base reference (`order`).
- **Class Association:** Modelling the interaction between a `customer` entity and dynamic order objects.

2. System Requirements

Component	Type	Minimum Specification	Recommended Specification
Hardware	Processor	Intel Core i3 / AMD Ryzen 3 (1.5 GHz)	Intel Core i5 / AMD Ryzen 5 (2.0 GHz+)
	RAM	1 GB	4 GB or higher
	Storage	200 MB free hard disk space	500 MB free SSD space
Software	Operating System	Windows 7 / Linux / macOS	Windows 10/11 or Ubuntu 20.04 LTS
	JDK Environment	Java Development Kit (JDK) 8	Java SE Development Kit 17 LTS or 21 LTS
	IDE / Editor	Command Line (<code>javac</code> / <code>java</code>)	Eclipse, IntelliJ IDEA, or VS Code

3. Class Diagram



4. Algorithm

Algorithm: Base Class Definition (order)

1. Define private fields: date and number.
2. Initialize state via parameterized constructor `order(date, number)`.
3. Define instance methods: `confirm()` to output confirmation status, and `close()` to indicate closure.

Algorithm: Subclasses Creation (specialorder & normalorder)

1. Create `specialorder` extending `order`; forward active parameter variables (date, number) to `super(date, number)`. Define specialized method `dispatch()`.
2. Create `normalorder` extending `order`; forward active parameters via `super(date, number)`. Define methods `dispatch()` and `receive()`.

Algorithm: Customer Interaction & Execution Flow

1. Define customer with attributes name and location. Create methods `sendorder(order)` and `receiveorder(order)`.
2. In `LabProgram4.main`, instantiate customer instance (e.g., "hemanth", "vizag").
3. Instantiate `specialorder("12th august", "44")` and process dispatch and receipt.
4. Instantiate `normalorder("13th august", "82")` and execute dispatch, customer receipt, and order closure workflows.

5. Compilation & Execution Instructions

Note: Save the source file with the exact name `LabProgram4.java` matching the public class name.

Step 1: Open Terminal / Command Prompt

```
cd path/to/your/java/files
```

Step 2: Compile the Program

```
javac LabProgram4.java
```

Step 3: Execute the Program

```
java LabProgram4
```

6. Refactored & Corrected Source Code

```
class order {
    private String date;
    private String number;

    public order(String date, String number) {
        this.date = date;
        this.number = number;
    }

    public void confirm() {
        System.out.println("order " + number + " confirmed on " + date);
    }

    public void close() {
        System.out.println("order " + number + " closed");
    }
}

class specialorder extends order {
    public specialorder(String date, String number) {
        super(date, number); // Fixed: passing dynamic parameters instead of literal strings
    }

    public void dispatch() {
        System.out.println("specialorder is dispatched");
    }
}

class normalorder extends order {
    public normalorder(String date, String number) {
        super(date, number); // Fixed: passing dynamic parameters instead of literal strings
    }

    public void dispatch() {
        System.out.println("normalorder is dispatched");
    }

    public void receive() {
        System.out.println("normalorder is received by customer");
    }
}
```

```

}

class customer {
    private String name;
    private String location;

    public customer(String name, String location) {
        this.name = name;
        this.location = location;
    }

    public void sendorder(order order) {
        System.out.println(name + " from " + location + " sent an order");
    }

    void receiveorder(order order) {
        System.out.println(name + " received order");
        order.close();
    }
}

public class LabProgram4 {
    public static void main(String[] args) {
        customer c = new customer("hemanth", "vizag");

        specialorder so = new specialorder("12th august", "44");
        c.sendorder(so);
        so.dispatch();
        c.receiveorder(so);

        System.out.println();

        normalorder no = new normalorder("13th august", "82");
        c.sendorder(no);
        no.dispatch();
        no.receive();
        c.receiveorder(no);
    }
}

```

7. Program Execution & Output

Terminal Execution Output

```

hemanth from vizag sent an order
specialorder is dispatched
hemanth received order
order 44 closed

hemanth from vizag sent an order
normalorder is dispatched
normalorder is received by customer
hemanth received order
order 82 closed

```